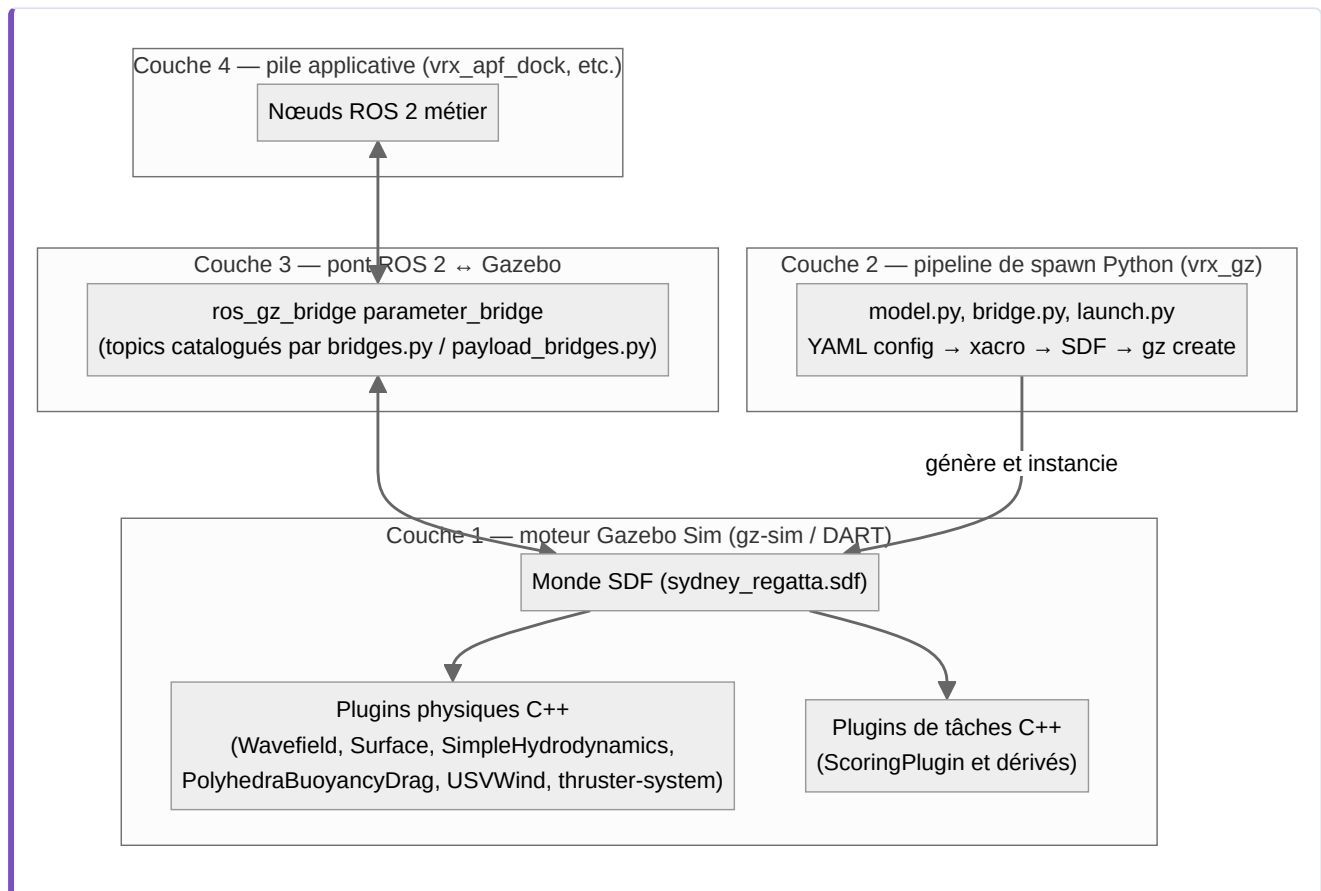


Anatomie de VRX / Gazebo

Environnement, physique, mécanique 3D, pipeline de spawn,
pont ROS 2 et tâches de compétition

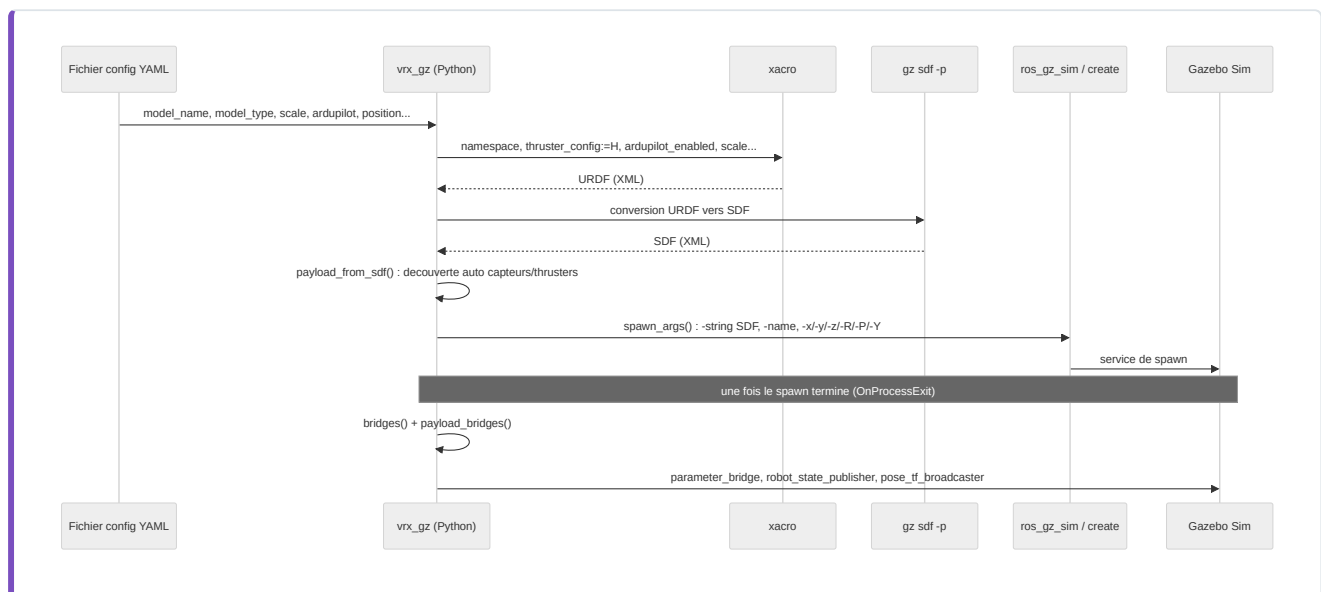
Yvan Eustache · 2026-09-03

1. Vue d'ensemble — quatre couches



- **Couche 1** : le monde SDF et les plugins C++ compilés (`libSurface.so` , `libUSVWind.so` , ...) — c'est du Gazebo Sim pur, aucune dépendance ROS.
- **Couche 2** : un ensemble de scripts Python (`vrx_gz/src/vrx_gz/`) qui **gènèrent** le SDF du véhicule à partir de xacro et **déclarent** les bridges — c'est de la génération de configuration, pas du run-time.
- **Couche 3** : `ros_gz_bridge` fait le pont topic-à-topic entre `gz-transport` et ROS 2 — c'est la seule couche que voit le code applicatif.
- **Couche 4** : votre code (`vrx_apf_dock` , etc.), documenté dans `ARCHITECTURE.md` .

2. Pipeline de spawn — d'un fichier YAML à un véhicule dans Gazebo



2.1 La classe `Model` (`vrx_gz/src/vrx_gz/model.py`)

C'est le cœur de tout le pipeline. Un objet `Model` porte `model_name`, `model_type` (`usv / wam-v` vs `vrx_hexrotor / vrx_quadrotor` → `is_UAV()` / `is_USV()`), `position`, `scale`, `ardupilot`, `payload`.

- `xacro_cmd()` construit l'appel `xacro` : `namespace:=<model_name>`, `locked:=true`, `imu_enabled:=true` (uniquement si `vrx_sensors_enabled:=false`, sinon la macro IMU serait instanciée deux fois — piège xacro documenté dans le code), `thruster_config:=H` toujours, y compris en mode ArduPilot (`ArduPilotPlugin` publie sur les mêmes topics que le thruster-system standard plutôt que de le remplacer, cf. [ARCHITECTURE.md §7.2](#)), `ground_truth_enabled:=true`, `ardupilot_enabled:=true` si applicable, `scale:=<scale>` pour les USV.
- `generate()` exécute xacro, convertit le résultat URDF en SDF via `gz sdf -p`, et appelle `payload_from_sdf()` : parcourt le SDF généré (via `sdformat14`) et **découvre automatiquement** tous les capteurs (`<sensor>`) et plugins connus (`gz::sim::systems::Thruster`, `JointPositionController`) pour peupler `self.payload` — c'est ce mécanisme qui permet à `vrx_apf_dock`, sans rien déclarer manuellement, de voir apparaître les bons bridges de capteurs.
- `spawn_args()` ne fait pas un appel `gz service` brut : c'est un nœud `ros_gz_sim / create` standard (`-string <SDF>`, `-name`, `-x/-y/-z/-R/-P/-Y`).
- `FromConfig(stream)` parse le YAML (`yaml.safe_load`), liste ou dict unique. Clés : `model_name`, `model_type` (requis), `position.xyz / rpy`, `scale`, `ardupilot` (USV), `flight_time`, `payload` (UAV).

2.2 `bridge.py` — la primitive de bridging

`Bridge(gz_topic, ros_topic, gz_type, ros_type, direction)`, `direction` ∈ {`BIDIRECTIONAL`, `GZ_TO_ROS`, `ROS_TO_GZ`} . `.argument()` génère l'argument CLI exact consommé par `ros_gz_bridge` `parameter_bridge` (ex. `/model/wamv/odometry@nav_msgs/msg/Odometry[gz.msgs.Odometry]`), `.remapping()` permet de renommer le topic côté ROS (c'est ainsi qu'un chemin gz-transport verbeux devient `sensors/imu/imu/data`).

2.3 `bridges.py` / `payload_bridges.py` — le catalogue

Deux familles de bridges :

Par véhicule (`Model.bridges()` , toujours présents) :

Bridge	Topic GZ	Topic ROS	Sens
<code>pose</code>	<code>/model/{m}/pose</code>	<code>pose</code> (<code>tf2_msgs/TFMessage</code>)	GZ → ROS
<code>pose_static</code>	<code>/model/{m}/pose_static</code>	<code>pose_static</code>	GZ → ROS
<code>joint_states</code>	<code>/world/{w}/model/{m}/joint_state</code>	<code>joint_states</code>	GZ → ROS
<code>cmd_vel</code> (UAV seulement)	<code>/model/{m}/cmd_vel</code>	<code>cmd_vel</code>	ROS → GZ
<code>magnetometer / air_pressure</code> (UAV)	<code>.../sensor/.../{type}</code>	<code>magnetic_field / air_pressure</code>	GZ → ROS

Par capteur/payload découvert (`payload_bridges()` , dispatché sur le type `sdf.Sensortype` ou une sous-chaîne du nom du plugin) :

Type détecté	Topic ROS	Message
<code>IMU</code>	<code>sensors/imu/imu/data</code>	<code>sensor_msgs/Imu</code>
<code>NAVSAT</code> (GPS)	<code>sensors/gps/gps/fix</code>	<code>sensor_msgs/NavSatFix</code>
<code>CAMERA / RGBD_CAMERA</code>	<code>sensors/cameras/{name}/image_raw</code> (+ <code>camera_info</code> , <code>depth</code>)	<code>sensor_msgs/Image</code> ...
<code>GPU_LIDAR</code>	<code>sensors/lidars/{name}/scan / points</code>	<code>LaserScan / PointCloud2</code>
<code>CONTACT</code>	<code>/vrx/contacts</code>	<code>ros_gz_interfaces/Contacts</code>
nom contient 'OdometryPublisher'	<code>sensors/position/ground_truth_odometry</code>	<code>nav_msgs/Odometry</code>
nom contient 'thruster_thrust_'	<code>thrusters/{side}/thrust</code>	<code>std_msgs/Float64</code> (ROS → GZ)
nom contient 'AcousticPinger'	<code>sensors/acoustics/receiver/range_bearing</code>	<code>ros_gz_interfaces/ParamVec</code>

Globaux/compétition (`competition_bridges()` , un seul `parameter_bridge` pour tous) : `clock` , `task_info` (`/vrx/task/info`), et selon le monde (listes `STATIONKEEPING_WORLDS` , `WILDLIFE_WORLDS` , etc.) les topics spécifiques à la tâche (`stationkeeping/goal` , `wildlife/animal{i}/pose` , `wayfinding/waypoints` ...).

⚠ Incohérence de nommage repérée : `AcousticTrackingScoringPlugin` publie par défaut sur `/vrx/acoustic_wayfinding/pose_error` (nom historique, côté C++), alors que `bridges.py` déclare `/vrx/acoustic_tracking/pose_error` côté ROS — les deux ne coïncident que si le monde SDF surcharge explicitement `<pose_error_topic>` sur le plugin. À vérifier avant de compter sur ce bridge.

2.4 Séquencement du lancement (`launch.py` , `competition.launch.py`)

`competition.launch.py` expose `world` , `sim_mode` (`full` / `sim` / `bridge`), `config_file` , `headless` , `competition_mode` (désactive les topics de debug comme le vent). Séquence : `simulation()` démarre `gz sim` (+ un processus `monitor_sim.py` qui est un **simple watchdog** — il fait `pgrep -f "gz sim"` toutes les secondes et déclenche l'arrêt de tout l'arbre de lancement ROS 2 quand Gazebo se termine, **ce n'est pas un moniteur de score**) → `spawn()` spawn le(s) modèle(s) puis, **après confirmation de fin du spawn** (`OnProcessExit`), lance le `parameter_bridge` + `robot_state_publisher` + `pose_tf_broadcaster` → `competition_bridges()` ajoute les topics globaux.

3. Modélisation de l'environnement

3.1 Le champ de vagues (`Wavefield.cc`) — pas un plugin, un moteur de calcul partagé

`vrx::Wavefield` n'est pas un `System` Gazebo : c'est une classe C++ instanciée à l'intérieur d'autres plugins (`Surface` , `PolyhedraBuoyancyDrag` , `WaveVisual`). Porté de `srmainwaring/asv_wave_sim` . Deux modes : lecture directe d'un bloc `<wavefield>` SDF, ou **abonnement à un topic** `gz.msgs.Param` (`/vrx/wavefield/parameters` par défaut) — c'est ce second mode qu'utilise `sydney_regatta.sdf` (voir §3.3).

Deux modèles de spectre (paramètre `model`) : - **PMS** (Pierson-Moskowitz, par défaut) : spectre océanique réaliste, $P_m(\omega, \omega_p) = \alpha \cdot g^2 / \omega^5 \cdot \exp(-5/4 \cdot (\omega_p / \omega)^4)$ avec $\alpha = 0.0081$. Les amplitudes des composantes sont échantillonnées autour de la fréquence moyenne. - **CWR** (Constant Wavelength-Amplitude Ratio) : composantes générées par un simple ratio d'échelle géométrique entre elles.

Hauteur d'eau en un point (x,y,t) — c'est la fonction appelée à **chaque pas de physique** par `Surface` et `PolyhedraBuoyancyDrag` :

$$h(x,y,t) = \sum_i a_i \cdot \cos(k_i \cdot (x \cdot dx_i + y \cdot dy_i) - \omega_i \cdot t) \cdot (1 - \exp(-(t-t_0)/\tau))$$

— une simple **superposition de sinusoides** (relation de dispersion en eau profonde $\omega = \sqrt{g \cdot k}$), avec une **montée en puissance exponentielle** au démarrage (τ , défaut 2 s) pour éviter un à-coup. Une seconde méthode (`ComputeDepthDirectly`) résout la **houle de Gerstner complète** (déplacement horizontal des particules, pas seulement vertical) par une résolution de Newton 2D — plus fidèle mais plus coûteuse, non utilisée dans la boucle physique par défaut.

Paramètres SDF clés : `number` (composantes, ≤ 3), `scale` (rapport plus grande/plus petite composante, 1.1), `angle` (étalement directionnel, 0.4 rad), `steepness` (cambrure Gerstner 0-1), `period` (5 s), `direction` , `gain` (multiplicateur d'amplitude PMS).

3.2 `WaveVisual` — le rendu, découplé de la physique

`WaveVisual` instancie **sa propre copie** de `Wavefield` (abonnée au même topic) uniquement pour extraire les paramètres numériques et les envoyer comme **uniforms de shader GPU** (Ogre2 : `Nwaves` , `amplitude[3]` , `wavenumber[3]` , `omega[3]` , `steepness[3]` ...) — le maillage océan à l'écran est

déformé sur GPU indépendamment du calcul physique CPU. Les deux restent cohérents parce qu'ils partagent le même flux de paramètres, mais **ce n'est pas la même exécution de code** — une divergence de shader n'affecterait jamais la physique, et vice-versa.

3.3 Configuration réelle dans `sydney_regatta.sdf`

- **Moteur physique** : `<physics type="dart">`, `max_step_size = 0.004 s` (250 Hz), `real_time_factor = 1.0`.
- **Plugins chargés au niveau du monde** : les systèmes `gz-sim-*-system` standards (physics, sensors avec `render_engine=ogre2`, imu, magnetometer, forcetorque, scene-broadcaster, contact, navsat, particle-emitter, user-commands) **plus** `libUSVWind.so` et `libPublisherPlugin.so` (celui-ci publie les paramètres du champ de vagues, voir ci-dessous). Aucun plugin de flottabilité au niveau monde — `Surface / SimpleHydrodynamics` viennent du SDF généré du WAM-V lui-même, `PolyhedraBuoyancyDrag` est attaché individuellement à chaque bouée `<include>`.
- **⚠ Par défaut, l'eau est calme et le vent est coupé** : `libPublisherPlugin.so` publie sur `/vrx/wavefield/parameters` avec `gain = 0.0` (donc amplitude de vague nulle, quel que soit `period / steepness` configurés), et `USVWind` est instancié avec `wind_mean_velocity = 0.0` et `var_wind_gain_constants = 0` (pas de rafales). Pour tester la robustesse de `vrx_apf_dock` en conditions dégradées, c'est **ici** qu'il faut agir (voir tableau §8).
- **Géoréférencement** : `<spherical_coordinates>` avec les coordonnées réelles du plan d'eau de Sydney Regatta (`-33.724223`, `150.679736`, WGS84/ENU) — c'est ce qui permet à un GPS simulé de renvoyer des coordonnées cohérentes.
- Les mondes `2023_practice/` et `2023_phase2/` (variantes numérotées par tâche : `wildlife0-5`, `stationkeeping0-5`, `scan_dock_deliver0-5` ...) partagent **exactement** ce même socle physique/environnement — seules les positions des accessoires/bouées et la config du plugin de scoring changent.

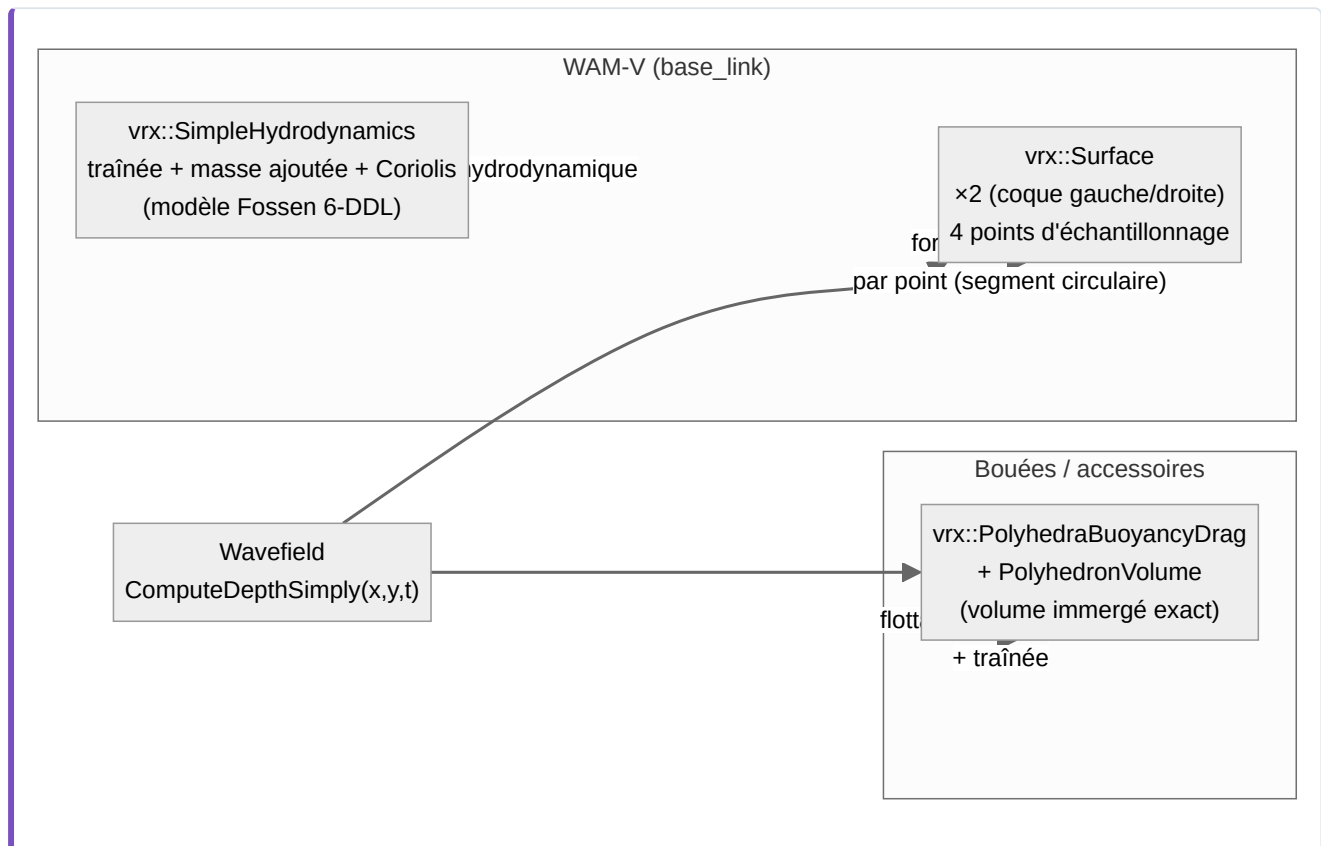
3.4 Le vent (`USVWind.cc`) — modèle de rafales de Gauss-Markov

Plugin **de niveau monde**, ciblant une liste de `<wind_obj>` (modèle + lien + `coeff_vector`). Modèle de rafale (bruit coloré, processus de type Ornstein-Uhlenbeck discrétisé) :

```
filterGain = gainConstant*sqrt(2*timeConstant)
varVel += (1/timeConstant)*(-varVel + filterGain/sqrt(dt)*N(0,1))*dt      # à chaque pas
velocity = varVel + windMeanVelocity
```

Puis force de traînée **quadratique**, signée : `Fx = coeff.x*relVel.x*|relVel.x|`, `Fy = coeff.y*relVel.y*|relVel.y|`, et un couple de lacet parasite `tau_z = -2*coeff.z*relVel.x*relVel.y` — un vent de travers génère donc aussi un couple de lacet, pas seulement une dérive latérale. Dans `sydney_regatta.sdf`, le seul objet vent configuré cible `wamv/base_link` avec `coeff_vector = (0.5, 0.5, 0.33)`.

4. Physique du véhicule — trois plugins, deux usages différents



4.1 Surface — flottabilité du WAM-V (approximation cylindrique par coque)

Chaque coque du catamaran est modélisée comme un **cylindre horizontal** (`hull_length = 4.9 · scale` m, `hull_radius = 0.213 · scale` m), échantillonné en **2 points** (avant/arrière) — soit **4 points au total** pour tout le WAM-V (2 coques × 2 points). Pour chaque point : profondeur d'eau locale via `Wavefield::ComputeDepthSimply`, puis force de flottabilité par l'aire du **segment circulaire** immergé :

```
CircleSegment(r,h) = r2 · acos((r-h)/r) - (r-h) · √(2rh - h2)
F = CircleSegment(hull_radius, deltaZ) · hull_length/nb_points · (-gravity.Z) · fluid_density
```

Toujours positive (poussée vers le haut uniquement) — c'est un modèle **simplifié**, sans traînée ni couple propre (la traînée/inertie vient du plugin suivant).

4.2 SimpleHydrodynamics — traînée, masse ajoutée, couplage de Coriolis

Modèle 6 degrés de liberté d'après **Fossen, *Guidance and Control of Ocean Vehicles***, appliqué au `base_link` entier. Trois contributions sommées en repère corps puis ramenées en repère monde :

- **Masse ajoutée** : $F_{am} = -M_a \cdot \text{stateDot}$ (Ma diagonale : `xDotU`, `yDotV`, `zDotW`, `kDotP`, `mDotQ`, `nDotR`).
- **Couplage de Coriolis** (termes croisés surge/sway ↔ lacet) : $C(0,5)=C(5,0)=yDotV \cdot v$, $C(1,5)=C(5,1)=xDotU \cdot u$.

- **Traînée linéaire + quadratique**, par axe : $D(i,i) = \text{paramLinéaire} + \text{paramQuad} \cdot |vitesse_i|$.

Valeurs réellement utilisées pour le WAM-V (`wamv_gazebo_dynamics_plugin.xacro`, mises à l'échelle en `scale2` — approximation « aire mouillée/frontale », pas un vrai scaling de Froude) :

Paramètre	Valeur (×scale ²)	Axe
<code>xU / xUU</code>	100 / 150	surge (avance)
<code>yV / yVV</code>	100 / 100	sway (dérive latérale)
<code>zW</code>	500	heave (pilonnement)
<code>kP / kPP</code>	300 / 600	roll
<code>mQ / mQQ</code>	900 / 900	pitch
<code>nR / nRR</code>	800 / 800	yaw (lacet — c'est ce terme qui amortit la rotation commandée par <code>thrust_mixer_node</code> / ArduPilot)

4.3 PolyhedraBuoyancyDrag — flottabilité exacte pour les bouées/accessoires

Contrairement à `Surface` (approximation cylindrique dédiée au WAM-V), ce plugin calcule le **volume immergé exact** d'un polyèdre (boîte/sphère/cylindre) contre le plan d'eau local, par découpage géométrique — méthode d'**Erin Catto** (*Exact Buoyancy for Polyhedra*, Game Programming Gems 6).

`PolyhedronVolume::SubmergedVolume()` triangule la forme, classe chaque face (immergée/sèche/coupée), et découpe les faces coupées en tétraèdres dont on somme les volumes signés (produit triple).

```
F_buoyancy = -fluid_density · volume_immergé · gravité
F_drag      = linear_drag · (masse · volume_immergé/volume_total) · (vitesse_eau - vitesse_objet)
```

Utilisé pour **toutes** les bouées de navigation (`mb_marker_buoy_*`, `mb_round_buoy_*`), pas pour le WAM-V.

5. Fichiers de définition mécanique 3D (xacro/URDF)

5.1 Coque — `wamv_base.urdf.xacro`

Un seul lien rigide `base_link` : visuel = maillage Collada unique, **collision décomposée** en primitives (6 cylindres de coque, 12 cylindres d'entretoises, 2 boîtes de jonction, 1 boîte de pont — bien moins coûteux pour le moteur physique qu'un maillage exact). Masse/inertie **mises à l'échelle selon les lois physiques dimensionnelles** :

```
masse      = 180 · scale3      (volume ~ échelle3, densité uniforme supposée)
ixx/iyy/izz = {120, 393, 446} · scale5  (inertie ~ masse × longueur2 ~ échelle5)
```

Cette même loi (`masse~scale3` , `inertie~scale5`) s'applique aux batteries (`23.5~scale3` kg), aux moteurs (`15~scale3` kg) et aux hélices (`0.5~scale3` kg) — c'est ce qui rend le « mini WAM-V » (`scale: 0.5` , utilisé pour le robot dans `two_wamv.yaml`) physiquement cohérent, pas juste visuellement réduit. `<enable_wind>true</enable_wind>` marque le lien comme sensible au plugin `USVWind` .

5.2 Batteries — `battery.xacro`

Masse ponctuelle articulée en joint fixe sur `base_link` (deux instances, `±1~scale` en y) — contribue à la masse/inertie totale du véhicule au niveau du moteur physique (les joints fixes sont fusionnés), sans lien physique dédié dans la logique de contrôle.

5.3 Layouts de propulseurs — `wamv_gazebo/urdf/thruster_layouts/`

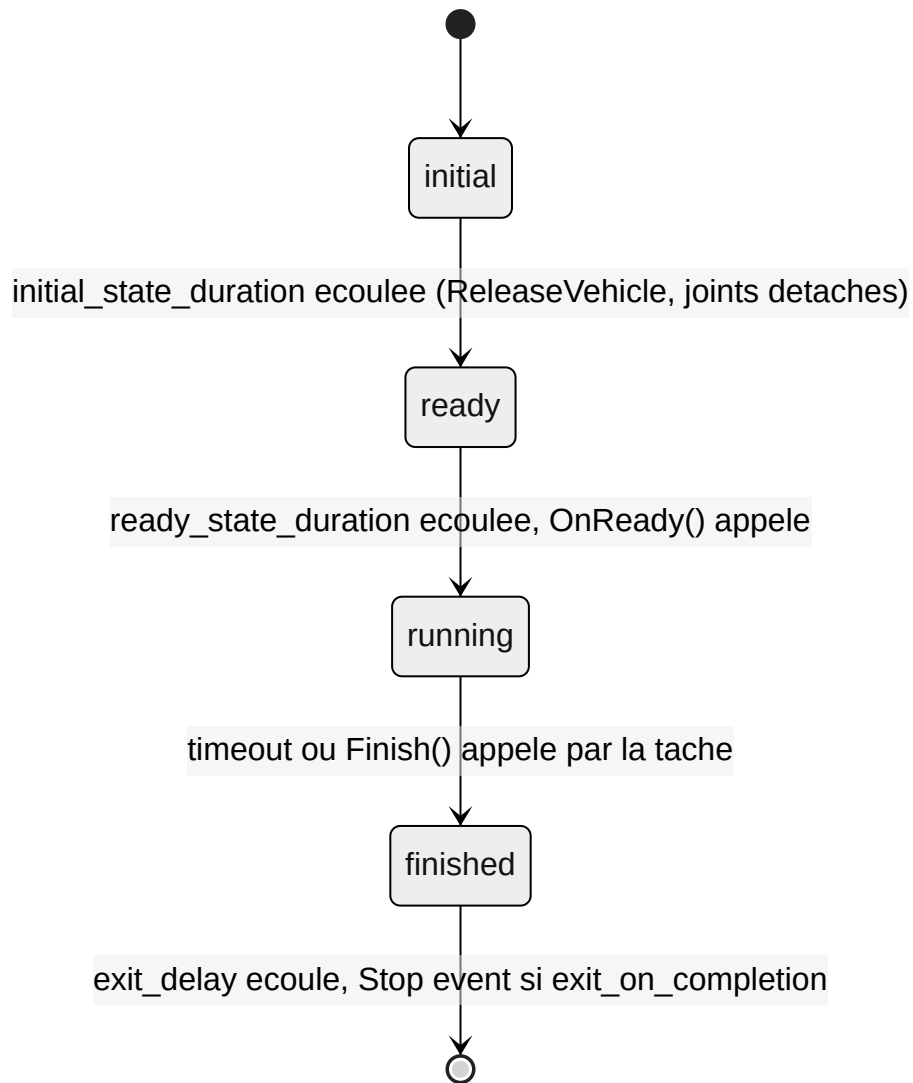
Layout	Propulseurs	Configuration physique
<code>H</code> (défaut)	2 arrière (<code>left / right</code>), fixes	Poussée différentielle pure — c'est le layout utilisé par <code>thrust_mixer_node</code> (§4.4 de <code>ARCHITECTURE.md</code>) et aussi par ArduPilot (<code>model.py</code> force <code>thruster_config:=H</code> même en mode ArduPilot)
<code>T</code>	2 arrière + 1 latéral (<code>yaw=90°</code>)	Ajoute une translation latérale directe
<code>X</code>	4 propulseurs à 45° (2 avant + 2 arrière)	Configuration vectorisée, contrôle quasi-holonyme (surge/sway/yaw indépendants)
ArduPilot (<code>wamv_aft_thrusters_ardupilot.xacro</code>)	Mêmes positions que <code>H</code> , sans le plugin <code>gz-sim-thruster-system</code>	Code mort actuellement : abandonné au profit du mode <code>COMMAND</code> d' <code>ArduPilotPlugin</code> qui réutilise le layout <code>H</code> standard (cf. <code>ARCHITECTURE.md</code> §7.2)

5.4 Moteur/hélice — `thrusters/engine.xacro`

Chaque propulseur = 2 liens (`{prefix}_engine_link` , `{prefix}_propeller_link`) + 2 joints : - `{prefix}_chassis_engine_joint` (revolute, axe Z, $\pm\pi$) — le joint d'**azimut/direction** (utilisé par les layouts vectorisés type `X`). - `{prefix}_engine_propeller_joint` (continuous, axe X, friction/damping 0.05) — le joint de **rotation de l'hélice**, celui que le plugin `gz-sim-thruster-system` commande réellement en vitesse angulaire pour produire la poussée (voir `wamv_gazebo_thruster_config.xacro` , déjà documenté dans `ARCHITECTURE.md` §4.5 / §7.2).

6. Plugins de tâches de compétition (scoring)

VRX est avant tout un simulateur de **compétition** (RobotX/RoboBoat) : chaque monde de tâche instancie un plugin dérivé de `ScoringPlugin` , qui implémente une machine à états commune.



Publication commune (1 Hz) sur `/vrx/task/info` (`gz.msgs.Param` → ROS `ros_gz_interfaces/ParamVec`): `state`, `elapsed_time`, `remaining_time`, `num_collisions`, `score`.
Les collisions sont comptées via un abonnement à `/vrx/contacts`, avec un `collision_buffer` anti-rebond.

Plugin	Tâche modélisée	Signal(aux) spécifique(s)
<code>StationkeepingScoringPlugin</code>	Maintenir position/cap sur un point fixe	<code>.../goal</code> , <code>.../pose_error</code> , <code>.../mean_pose_error</code>
<code>NavigationScoringPlugin</code>	Traverser un chenal de portes (paires de bouées)	Points par porte + bonus d'enchaînement, pénalité de collision
<code>WildlifeScoringPlugin</code>	Éviter/contourner des bouées-animaux (sens horaire/anti-horaire)	<code>.../animal{i}/pose</code> (1 Hz)
<code>GymkhanaScoringPlugin</code>	Composite : chenal + recherche par pinger acoustique	Délègue à des instances internes de <code>NavigationScoringPlugin</code> / <code>StationkeepingScoringPlugin</code>
<code>ScanDockScoringPlugin</code>	Lire une séquence de couleurs, docker dans la bonne baie, tirer une balle	<code>color_sequence</code> (ROS → GZ, soumission du concurrent), <code>symbol_topic</code> par baie
<code>PerceptionScoringPlugin</code>	Identifier/localiser des objets apparus en scène	<code>landmark</code> (ROS → GZ, soumission du concurrent)
<code>AcousticTrackingScoringPlugin</code>	Suivre un pinger acoustique mobile	Même schéma pose-erreur que Stationkeeping, mais cible mobile (⚠ voir incohérence de topic §2.3)
<code>AcousticPerceptionScoringPlugin</code>	Homing vers un pinger fixe (tolérance 1 m)	S'appuie sur <code>AcousticPingerPlugin</code>

Plugins auxiliaires (accessoires, pas de scoring propre) : `AcousticPingerPlugin` (capteur portée/relèvement bruité), `LightBuoyPlugin` / `PlacardPlugin` (affichage couleur/symbole), `BallShooterPlugin` (lanceur à ressort, 250 N par défaut), `WaypointMarkers` (marqueurs de visualisation Gazebo, même mécanisme `/marker_array` que `vector_marker_node` — voir `ARCHITECTURE.md` §4.5).

Pertinence pour `vrx_apf_dock` : ces plugins ne sont pas utilisés par le projet de docking actuel, mais `StationkeepingScoringPlugin` en particulier est un bon gabarit si l'on veut un jour **chronométrer/scorer automatiquement** un essai de docking (temps, précision finale, nombre de collisions) plutôt que de le juger à l'œil en simulation.

7. Liens avec ROS 2 et ArduPilot — synthèse

- **ROS 2** : jamais de dépendance directe entre les plugins C++ Gazebo et ROS — tout passe par `ros_gz_bridge` (couche 3, §1), déclaré dynamiquement par les scripts Python de la couche 2. C'est pour cela qu'ajouter un capteur au WAM-V (un xacro de plus dans `wamv_gazebo.urdf.xacro`) suffit à le faire apparaître côté ROS **sans toucher au pipeline de spawn** : `payload_from_sdf()` le détecte automatiquement.
- **ArduPilot** : ne passe pas par cette couche de bridging du tout — `ArduPilotPlugin` (package `ardupilot_gazebo`, documenté dans `ARCHITECTURE.md` §7) communique en JSON/UDP direct avec Gazebo (lien FDM), et publie sa commande de poussée en mode `COMMAND` sur les **mêmes topics gz-transport natifs** que le plugin `gz-sim-thruster-system` (`wamv/thrusters/{left,right}/thrust`) — du point de vue de VRX, ArduPilot est indiscernable d'un nœud ROS qui publierait sur ces topics.
- **Environnement (vagues/vent)** : aujourd'hui **désactivé** dans le monde utilisé par `vrx_apf_dock` / `vrx_ardupilot` (`gain=0`, `wind_mean_velocity=0`). C'est un choix de simplification pour le développement du docking, mais cela signifie que **la robustesse de la loi APF aux perturbations n'a jamais été testée en simulation** — voir chapitre proposé §9.3.

8. Paramètres clés à modifier — aide-mémoire pratique

Je veux...	Fichier	Paramètre
Activer de la houle	<code>sydney_regatta.sdf</code> (bloc <code>PublisherPlugin</code>)	<code>gain</code> (0 → ex. 0.5), <code>period</code> , <code>steepness</code>
Activer du vent avec rafales	<code>sydney_regatta.sdf</code> (bloc <code>USVWind</code>)	<code>wind_mean_velocity</code> , <code>var_wind_gain_constants</code>
Changer la taille du WAM-V	fichier de config de spawn (ex. <code>two_wamv.yaml</code>)	<code>scale</code>
Changer le nombre/agencement des propulseurs	<code>wamv_gazebo.urdf.xacro</code> (arg passé par <code>model.py</code>)	<code>thruster_config</code> (H / T / X)
Régler l'amortissement en lacet (yaw) du bateau	<code>wamv_gazebo_dynamics_plugin.xacro</code>	<code>nR</code> / <code>nRR</code> (<code>SimpleHydrodynamics</code>)
Réduire le pas de physique (précision ↑, coût CPU ↑)	<code>sydney_regatta.sdf</code> (<code><physics></code>)	<code>max_step_size</code> (défaut 0.004 s)
Accélérer/ralentir la simulation par rapport au temps réel	<code>sydney_regatta.sdf</code> (<code><physics></code>)	<code>real_time_factor</code>
Ajouter un capteur (caméra, lidar...) au WAM-V	xacro de capteur (<code>wamv_gazebo/urdf/components/</code>) inclus dans <code>wamv_gazebo.urdf.xacro</code>	— (bridge auto-découvert, voir §2.1)
Lancer sans rendu (calcul plus rapide, pas de GUI/markers)	argument de lancement	<code>headless:=true</code>
Scorer automatiquement un run (temps, précision, collisions)	nouveau plugin inspiré de <code>StationkeepingScoringPlugin</code>	<code><running_state_duration></code> , <code><collision_buffer></code>

9. Chapitres additionnels proposés

Ce document couvre le socle demandé ; plusieurs sujets connexes mériteraient un chapitre dédié si le besoin se présente :

1. **Génération procédurale des mondes de tâche** (`2023_phase2/` , `2023_practice/`) — comment les 6 variantes numérotées par tâche sont dérivées (positions de bouées, seed aléatoire), et comment en générer une nouvelle pour un scénario de docking spécifique.
2. **Performance et temps réel** — coût du pas de physique à 250 Hz, impact du rendu Ogre2/ `headless` , comment profiler un monde qui tourne sous le `real_time_factor` cible (pertinent avant tout essai HIL, cf. `ARCHITECTURE_HARDWARE.md` phase 1).
3. **Robustesse aux perturbations environnementales** — activer vent/houle (§8) et quantifier la dégradation de la précision de docking de `apf_lib.py` , aujourd'hui jamais testée puisque l'environnement par défaut est calme.
4. **Ajouter un nouveau type de véhicule** — ce que `is_UAV()` / `is_USV()` et le pipeline de spawn supposent en dur, et ce qu'il faudrait généraliser pour un véhicule hors catalogue (ex. un ROV).
5. **Debug et introspection Gazebo** — `gz topic -e / gz service -l` , lecture des marqueurs `/marker_array` , comment tracer une force appliquée par un plugin (`SimpleHydrodynamics` , `USVWind`) pour diagnostiquer un comportement physique inattendu.
6. **Créer un nouveau plan d'eau géoréférencé** — utiliser `<spherical_coordinates>` pour faire correspondre un monde Gazebo à un vrai plan d'eau (utile en amont d'un essai réel, cf. `ARCHITECTURE_HARDWARE.md` §9 phase 2/3).
7. **Le pont gz-transport natif vs `ros_gz_bridge`** — pourquoi `vector_marker_node` / `apf_field_marker_node` contournent délibérément le bridge (service `/marker_array` , sans équivalent ROS), et dans quels autres cas parler gz-transport directement est préférable à passer par ROS 2.
8. **Tests et CI du dépôt VRX amont** (`.github/workflows`) — ce qui est testé automatiquement en amont, utile pour juger si un patch local (cf. `ARCHITECTURE.md` §7.2) risque de casser au prochain `git pull` upstream.